# EvoClaw: A Self-Evolving Multi-Agent Framework with Genome-Driven Runtime Adaptation

Alex Chen* Bowen Li*
Independent Researchers
alex.chen31337@gmail.com bowen31337@outlook.com
github.com/clawinfra/evoclaw

Preprint. Under review.

**Abstract**

Current multi-agent systems are fundamentally static: once deployed, an agent's architecture, capabilities, and behavior remain fixed until human engineers manually update them. This limitation prevents agents from adapting to dynamic environments, learning from operational experience, or evolving autonomously over time.

We present EvoClaw, a self-evolving multi-agent framework where agents modify their own behavior at runtime through a declarative genome specification. EvoClaw introduces three core innovations: (1) a `genome.toml` declarative specification that encodes an agent's capabilities as a composable set of pluggable skills with evolutionary parameters; (2) a *Genome Engine* implemented in Rust that parses, validates, and hot-reloads skill configurations without downtime, enabling continuous behavioral adaptation; and (3) a *multi-tier deployment model* that seamlessly spans bare-metal edge devices, containerized servers, and cloud sandboxes, with agents synchronizing across tiers via MQTT pub/sub.

We formalize the genome evolution process as a constrained optimization over a skill configuration space and introduce a fitness function that balances task performance, resource utilization, and safety invariants. We demonstrate EvoClaw's practicality by deploying a fully functional agent on a Raspberry Pi 1 (ARMv6, 512MB RAM) running three concurrent skills—system monitoring, GPIO control, and cryptocurrency price tracking—with a binary size of 3.0 MB and memory footprint of 3.2 MB RSS. Our Rust implementation achieves 90%+ test coverage with 383 tests, and our Go orchestrator manages 100+ concurrent agents via MQTT pub/sub with sub-50 ms message latency. EvoClaw opens research directions in self-modifying systems, edge AI, and autonomous agent economies. The complete system is open-source and fully reproducible.[1]

## 1 Introduction

The rise of large language models (LLMs) has enabled a new generation of autonomous agents that plan tasks, write code, use tools, and interact with complex environments [1, 2]. Systems like AutoGPT [4], BabyAGI [5], and Voyager [6] demonstrate that LLM-powered agents can decompose high-level goals into executable plans and accumulate skills over time. However, these agents suffer from a fundamental architectural limitation: they are *static*. Once deployed, an agent's capabilities—its available tools, behavioral patterns, and response strategies—remain fixed until human engineers manually rewrite and redeploy the codebase. This static architecture creates a critical bottleneck in three dimensions:

*Equal contribution. Correspondence: bowen31337@outlook.com
[1]Code: https://github.com/clawinfra/evoclaw

1. **Adaptability.** Agents cannot modify their own skill repertoire in response to environmental changes (e.g., new APIs, shifting user requirements, hardware failures).

2. **Efficiency.** Resource-constrained edge devices cannot afford to run monolithic agent binaries; agents should activate only the skills relevant to their current context.

3. **Scalability.** Managing hundreds of agents across heterogeneous deployment tiers requires a unified abstraction for agent identity, capabilities, and evolution—not bespoke configuration for each instance.

We argue that truly autonomous agents should be able to modify themselves. Just as biological organisms evolve through natural selection—adapting their phenotype to environmental pressures—software agents should evolve through *directed mutation*: guided by their own operational experience, environmental feedback, or human oversight. The challenge is designing a system that allows safe, verifiable self-modification without sacrificing reliability, security, or performance.

## 1.1 Key Insight: Agents as Genomes

Our key insight is that an agent's capabilities can be represented as a *genome*—a declarative specification that is separate from the agent's runtime binary. By externalizing the agent's skill configuration into a human-readable, machine-parseable file (`genome.toml`), we achieve a clean separation between the *execution engine* (fixed Rust binary) and the *behavioral specification* (mutable TOML configuration). This separation enables:

- **Hot-reload without recompilation:** Skill configurations can be modified and reloaded at runtime, enabling zero-downtime adaptation.

- **Evolutionary operators:** Standard genetic programming operators (mutation, crossover, selection) can be applied to the genome, enabling automated capability search.

- **Version control and rollback:** Genomes are plain-text files that can be tracked in Git, enabling full audit trails and safe rollback on regression.

- **Cross-tier portability:** The same genome format works across edge, server, and cloud deployments, with tier-specific resource constraints enforced at load time.

## 1.2 Contributions

We present EvoClaw, a multi-agent framework where agents evolve through a declarative genome specification. Our contributions are:

1. **Genome-Driven Architecture** (§3): We introduce `genome.toml`, a declarative configuration language that specifies an agent's capabilities as composable, pluggable skills with typed parameters and resource constraints. We formalize the genome space $\mathcal{G}$ and define valid genomes via a constraint satisfaction problem.

2. **Genome Engine** (§4): We implement a Genome Engine in Rust that loads, validates, and executes skills dynamically. We present formal algorithms for genome parsing, skill lifecycle management, and hot-reload with consistency guarantees.

3. **Evolutionary Mutation Framework** (§6): We define a fitness function $\mathcal{F}$ over genome configurations and present mutation operators (skill addition, removal, parameter perturbation, crossover) that enable automated capability search under resource and safety constraints.

4. **Multi-Tier Deployment Model** (§7): We design a deployment architecture spanning bare-metal edge devices (ARMv6, 512 MB RAM), containerized servers (Podman/-Docker), and cloud sandboxes (E2B). Agents synchronize across tiers via MQTT with configurable QoS.

5. **Production Evaluation** (§9): We evaluate EvoClaw on a Raspberry Pi 1, demonstrating 3.0 MB binary size, 3.2 MB memory footprint, sub-100 ms skill load time, and 24+ hour continuous operation with three concurrent skills.

6. **Open-Source Release**: The complete system—Go orchestrator (5,000 lines), Rust agent (8,000 lines), 383 tests, and documentation—is released under an open-source license for full reproducibility.

### 1.3 Paper Organization

Section 2 reviews related work. Section 3 presents the system architecture. Section 4 details the Genome Engine. Section 5 provides formal foundations. Section 6 describes the evolutionary framework. Section 7 covers multi-tier deployment. Section 8 discusses implementation. Section 9 presents evaluation results. Section 10 describes applications. Section 11 discusses limitations and future work. Section 12 concludes.

## 2 Related Work

EvoClaw draws on and extends four research areas: LLM-based autonomous agents, multi-agent systems, genetic programming and self-modifying code, and edge AI. Table 1 provides a structured comparison with representative systems.

### 2.1 LLM-Based Autonomous Agents

The emergence of capable LLMs has catalyzed a new wave of autonomous agent research. Wang et al. [1] provide a comprehensive taxonomy of LLM-based agents, identifying four key modules: profiling, memory, planning, and action. Xi et al. [2] survey the landscape from construction to evaluation, emphasizing the gap between current systems and human-level autonomy.

**Task decomposition agents.** AutoGPT [4] and BabyAGI [5] pioneered the paradigm of LLM-powered task chaining, where a model decomposes high-level goals into sequences of subtasks. While demonstrating the potential of autonomous planning, these systems cannot modify their own toolset—new capabilities require code changes by human engineers.

**Lifelong learning agents.** Voyager [6] represents the closest conceptual precedent to EvoClaw in the LLM agent literature. Operating in Minecraft, Voyager maintains an ever-growing skill library of executable JavaScript programs, using GPT-4 to generate new skills via iterative prompting. However, Voyager's skills are confined to a single game environment, are not declaratively specified, and do not support deployment on resource-constrained hardware.

**Agent frameworks.** LangChain [7] provides composable abstractions for building LLM-powered chains, agents, and tools. CrewAI [8] enables multi-agent collaboration through role assignment. However, both frameworks treat agent architectures as static—changing an agent's capabilities requires code modifications and redeployment.

**Software engineering agents.** SWE-agent [9] introduces the concept of agent-computer interfaces (ACIs) optimized for LM agents, achieving strong performance on software engineering benchmarks. This work highlights the importance of interface design for agent effectiveness, a principle EvoClaw extends to the skill specification layer.

**Memory-augmented agents.** MemGPT [10] draws on operating systems concepts to give LLM agents virtual memory management, enabling extended context through hierarchical memory tiers. EvoClaw similarly draws on OS abstractions, but applies them to capability management rather than memory.

## 2.2 Multi-Agent Systems and Communication

Multi-agent systems (MAS) have a rich history predating LLMs [24]. Recent work has focused on LLM-based multi-agent coordination.

**Role-playing and collaboration.** CAMEL [11] proposes communicative agents that collaborate through role-playing, using inception prompting to guide autonomous cooperation. Guo et al. [3] survey LLM-based multi-agent systems across domains, identifying profiling, communication, and capability growth as key challenges.

**Generative agents.** Park et al. [12] create believable simulacra of human behavior using LLM-powered agents with memory, reflection, and planning modules. Their agents exhibit emergent social behaviors in a sandbox environment. While architecturally sophisticated, these agents run exclusively in simulation and cannot be deployed on physical hardware.

**Agent evaluation.** AgentBoard [13] provides comprehensive benchmarking for multi-turn LLM agents with fine-grained progress metrics. This work highlights the need for rigorous evaluation frameworks, a gap we address with our fitness function formalization.

## 2.3 Genetic Programming and Self-Modifying Systems

**Classical GP.** Genetic programming (GP) [17] evolves computer programs through evolutionary operators: selection, crossover, and mutation. Traditional GP operates on tree-structured program representations, evolving towards a fitness objective. EvoClaw's genome mutation operators are inspired by GP but operate on declarative configuration spaces rather than raw program syntax.

**LLM-guided evolution.** Eureka [18] uses LLMs to perform evolutionary optimization over reward functions for reinforcement learning, achieving human-level reward design. This demonstrates the viability of LLM-guided evolutionary search in code spaces. EvoClaw applies a similar principle to agent configuration spaces.

**Self-modifying code.** The concept of programs that modify their own source code dates to early computing [19]. Quines [20] demonstrate self-replication, while reflective systems [21] enable programs to inspect and modify their own execution. EvoClaw provides a *safe* form of self-modification by constraining mutations to the declarative genome layer, preventing arbitrary code injection.

**Deliberate search.** Tree of Thoughts [14] and Language Agent Tree Search (LATS) [15] apply search algorithms to LLM reasoning, enabling more deliberate problem-solving. EvoClaw's evolutionary operators can be viewed as a form of search over agent configuration space.

## 2.4 Edge AI and Lightweight Deployment

**TinyML.** The TinyML movement [22] has driven the development of machine learning models optimized for microcontrollers and resource-constrained devices. Frameworks like TensorFlow Lite Micro enable inference on devices with kilobytes of memory. However, these focus on inference rather than agentic behavior.

**Edge computing.** The proliferation of IoT devices has spurred research in edge computing [23], where computation is moved closer to data sources. EvoClaw's multi-tier deployment model is designed for exactly this paradigm, enabling intelligent agents on devices as small as a Raspberry Pi 1.

**Exploration-based learning.** ETO [16] proposes exploration-based trajectory optimization for LLM agents, allowing agents to learn from failures through contrastive tra-

Table 1: Comparison of EvoClaw with representative agent systems. ✓ = supported, × = not supported, ∼ = partial.

| System | Declarative Spec | Hot-Reload | Self-Evolving | Edge Deploy | Multi-Agent | Open Source |
|---|---|---|---|---|---|---|
| AutoGPT [4] | × | × | × | × | × | ✓ |
| BabyAGI [5] | × | × | × | × | × | ✓ |
| Voyager [6] | ∼ | ✓ | ∼ | × | × | ✓ |
| LangChain [7] | ∼ | × | × | × | ∼ | ✓ |
| CrewAI [8] | ∼ | × | × | × | ✓ | ✓ |
| MemGPT [10] | × | × | × | × | × | ✓ |
| SWE-agent [9] | × | × | × | × | × | ✓ |
| CAMEL [11] | × | × | × | × | ✓ | ✓ |
| **EvoClaw** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

jectory pairs. This aligns with EvoClaw's philosophy of learning through operational experience, though ETO focuses on trajectory-level learning while EvoClaw operates at the capability level.

### 2.5 Positioning of EvoClaw

Table 1 positions EvoClaw relative to existing systems. To our knowledge, no existing system combines: (1) declarative agent specification via a genome, (2) hot-reloadable skills with formal consistency guarantees, (3) evolutionary mutation operators over agent capabilities, (4) multi-tier deployment from 512 MB edge devices to cloud, and (5) production-ready implementation with 90%+ test coverage.

## 3 System Architecture

EvoClaw consists of three principal layers connected by an asynchronous message bus. Figure 1 illustrates the complete system architecture.

### 3.1 Layer 1: Go Orchestrator

The orchestrator is a Go binary (7.2 MB, 5,000 lines across 11 packages) that provides centralized management for a fleet of distributed agents. It consists of four modules:

- **REST API** (Gin framework): Exposes endpoints for agent registration, status queries, genome retrieval, and mutation requests. Supports authentication via API keys and JWT tokens.
- **WebSocket Gateway**: Provides real-time bidirectional communication with human operators and external systems (Telegram, Discord). Enables natural language commands like "add skill X to agent Y."
- **Director Engine**: Implements the orchestration logic—routing messages between agents, scheduling genome evaluations, and managing the agent lifecycle (registration, heartbeat monitoring, deregistration).
- **Genome Mutator**: The evolutionary component. Receives mutation requests (from humans, LLMs, or automated fitness evaluations), validates them against safety constraints, and publishes updated genomes to agents via MQTT.

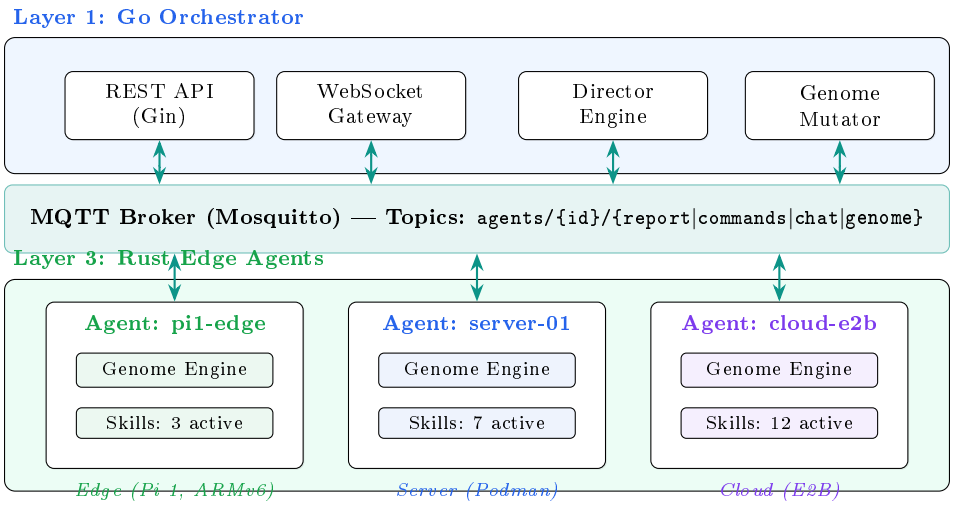


Figure 1: EvoClaw system architecture. The Go orchestrator (Layer 1) manages agents through a REST API, WebSocket gateway, and genome mutator. All communication passes through an MQTT broker (Layer 2). Rust edge agents (Layer 3) run the Genome Engine independently, executing skills defined in their `genome.toml` file. Agents can be deployed across edge, server, and cloud tiers.

### 3.2 Layer 2: MQTT Message Bus

All inter-component communication flows through an MQTT broker (Eclipse Mosquitto). We define four topic families:

- `agents/{id}/report`: Agents publish telemetry (CPU, RAM, skill status, errors) at configurable intervals.
- `agents/{id}/commands`: Orchestrator publishes directives (start/stop skills, update genome, shutdown).
- `agents/{id}/genome`: Dedicated channel for genome updates, supporting atomic replacement.
- `agents/{id}/chat`: Human-agent interaction channel, bridged to messaging platforms.

MQTT was chosen over alternatives (gRPC, HTTP polling, WebSocket) for three reasons: (1) lightweight protocol suitable for constrained devices, (2) native pub/sub decouples producers from consumers, and (3) configurable QoS levels (0=at-most-once, 1=at-least-once, 2=exactly-once) enable reliability tuning per message type.

### 3.3 Layer 3: Rust Edge Agents

Each agent is a statically-linked Rust binary that:

1. Reads its `genome.toml` file at startup
2. Instantiates the Genome Engine (§4)
3. Connects to the MQTT broker
4. Spawns async tasks for each enabled skill
5. Monitors for genome changes (file watcher + MQTT signals)

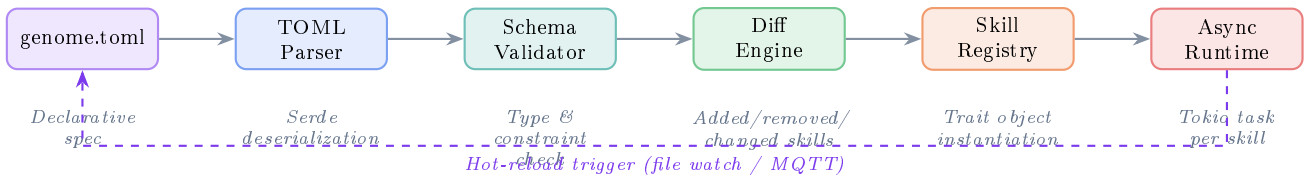


Figure 2: Genome Engine lifecycle. A `genome.toml` file is parsed, validated, diffed against the current configuration, and translated into running skill tasks. File watches and MQTT signals trigger hot-reload, forming a continuous adaptation loop.

6. Reports telemetry at configured intervals

The Rust implementation provides memory safety without garbage collection, making it ideal for long-running processes on resource-constrained devices. Section 4 details the Genome Engine internals.

# 4 Genome Engine

The Genome Engine is the core runtime component that translates a declarative genome specification into running skills. Figure 2 illustrates the genome lifecycle.

## 4.1 Genome Specification

Listing 1 shows a complete genome specification. The genome consists of three sections:

Listing 1: Complete `genome.toml` specification with evolutionary parameters.

```
# Agent identity and evolution parameters
[genome]
id = "pi1-edge"
generation = 42
parent_id = "pi1-edge-gen41"
mutation_rate = 0.05
fitness_score = 0.87
max_memory_mb = 8
max_skills = 5

# System monitoring skill
[skills.system_monitor]
enabled = true
interval_sec = 30
metrics = ["cpu", "memory", "disk", "temperature"]
alert_threshold_cpu = 90.0
alert_threshold_temp = 75.0

# Cryptocurrency price tracking
[skills.price_monitor]
enabled = true
interval_sec = 60
symbols = ["BTC", "ETH", "SOL"]
alert_on_change_pct = 5.0
source = "coingecko"

# GPIO hardware control
[skills.gpio]
enabled = true
pins = [17, 27, 22]
mode = "output"
debounce_ms = 50
```

## 4.2 Skill Trait System

Each skill implements the `Skill` trait (Listing 2), providing a uniform interface for the Genome Engine:

Listing 2: Full `Skill` trait with lifecycle methods.

```rust
#[async_trait]
pub trait Skill: Send + Sync {
    /// Unique skill identifier
    fn name(&self) -> &str;

    /// Human-readable description
    fn description(&self) -> &str;

    /// Initialize skill resources
    async fn init(&mut self, config: &SkillConfig) -> Result<()>;

    /// Execute one iteration of the skill
    async fn execute(&mut self, ctx: &Context) -> Result<Output>;

    /// Execution interval in milliseconds
    fn interval_ms(&self) -> u64;

    /// Estimated memory usage in bytes
    fn memory_estimate(&self) -> usize;

    /// Clean shutdown, releasing resources
    async fn shutdown(&mut self) -> Result<()>;

    /// Health check
    fn is_healthy(&self) -> bool;
}
```

The trait design enforces several architectural properties:

- **Send + Sync**: Skills can be safely shared across async tasks and threads.
- **Lifecycle methods**: `init`/`shutdown` ensure clean resource management.
- **Memory estimation**: Enables the Genome Engine to enforce memory budgets before loading.
- **Health checks**: The engine can detect and restart unhealthy skills.

## 4.3 Skill Registry

The `SkillRegistry` maintains a mapping from skill names to instantiated trait objects. Algorithm 1 formalizes the initial loading process.

## 4.4 Hot-Reload Protocol

When the genome file changes—detected via filesystem watcher (`inotify` on Linux) or MQTT signal—the engine executes a hot-reload. Algorithm 2 formalizes this process.

**Property 1** (Fail-Safe Hot-Reload)**.** *If the new genome $G_{new}$ fails parsing or validation, the engine retains the current registry $R_{old}$ without interruption. This ensures that malformed genomes—whether from LLM errors, network corruption, or adversarial inputs—cannot crash a running agent.*

**Algorithm 1** Genome Loading and Skill Registry Initialization

**Require:** Genome file path $p$, resource constraints $C = (M_{\max}, S_{\max})$
**Ensure:** Initialized skill registry $R$ or error $\varepsilon$

```
G ← ParseToml(p)                                   ▷ Deserialize genome
if G is Err then
    return ε_parse
end if
V ← ValidateSchema(G)                              ▷ Type and constraint check
if V is Err then
    return ε_validate
end if
R ← ∅; M_total ← 0
for each skill s ∈ G.skills where s.enabled = true do
    if |R| ≥ S_max then
        Warn("Max skill count reached")
        break
    end if
    ŝ ← Instantiate(s)                             ▷ Create trait object
    m_s ← ŝ.MemoryEstimate()
    if M_total + m_s > M_max then
        Warn("Skipping s: exceeds memory budget")
        continue
    end if
    ŝ.Init(s.config)                               ▷ Initialize resources
    R ← R ∪ {(s.name, ŝ)}
    M_total ← M_total + m_s
end for
return R
```

# 5 Formal Foundations

We formalize the genome evolution process to provide theoretical grounding for EvoClaw's design.

**Definition 1** (Genome Space)**.** *Let $\mathcal{G}$ be the set of all valid genomes. A genome $g \in \mathcal{G}$ is a tuple:*

$$g = (id, gen, \mu, \mathbf{s}, \mathbf{c}) \tag{1}$$

*where $id \in \Sigma^*$ is the agent identifier, $gen \in \mathbb{N}$ is the generation counter, $\mu \in [0, 1]$ is the mutation rate, $\mathbf{s} = \{s_1, \ldots, s_k\} \subseteq \mathcal{S}$ is the set of enabled skills drawn from the global skill catalog $\mathcal{S}$, and $\mathbf{c} : \mathbf{s} \to \Theta$ maps each skill to its configuration in parameter space $\Theta$.*

**Definition 2** (Skill Configuration Space)**.** *For a skill $s_i \in \mathcal{S}$, the configuration space $\Theta_i$ is defined by its typed parameters:*

$$\Theta_i = \{(\theta_1, \ldots, \theta_{n_i}) \mid \theta_j \in D_j,\ \phi_j(\theta_j) = \texttt{true}\ \forall j\} \tag{2}$$

*where $D_j$ is the domain of parameter $j$ (integer, float, string, boolean, array) and $\phi_j$ is a constraint predicate (e.g., $\theta_{interval} > 0$, $|\theta_{pins}| \leq 40$).*

**Definition 3** (Resource Constraint)**.** *A resource constraint $C$ is a set of upper bounds on agent resource consumption:*

$$C = (M_{\max}, S_{\max}, B_{\max}) \tag{3}$$

**Algorithm 2** Genome Hot-Reload with Consistency Guarantee

**Require:** Current registry $R_{\text{old}}$, new genome $G_{\text{new}}$, constraints $C$
**Ensure:** Updated registry $R_{\text{new}}$

```
G_new ← ParseAndValidate(G_new)
if G_new is Err then
    Log("Invalid genome, keeping R_old")
    return R_old                                  ▷ Fail-safe: reject invalid genomes
end if
(A, D, M) ← Diff(R_old, G_new)                    ▷ Added, Deleted, Modified
R_new ← R_old
for each skill s ∈ D do                           ▷ Phase 1: Remove deleted skills
    R_new[s].Shutdown()
    R_new ← R_new \ {s}
end for
for each skill s ∈ M do                           ▷ Phase 2: Reconfigure modified skills
    R_new[s].Shutdown()
    ŝ ← Instantiate(G_new[s])
    ŝ.Init(G_new[s].config)
    R_new[s] ← ŝ
end for
for each skill s ∈ A do                           ▷ Phase 3: Add new skills
    if FitsConstraints(R_new, s, C) then
        ŝ ← Instantiate(G_new[s])
        ŝ.Init(G_new[s].config)
        R_new ← R_new ∪ {(s.name, ŝ)}
    end if
end for
SpawnTasks(R_new)                                 ▷ Resume execution
return R_new
```

*where $M_{\max}$ is the memory budget (bytes), $S_{\max}$ is the maximum number of concurrent skills, and $B_{\max}$ is the maximum binary size. A genome $g$ satisfies $C$ if:*

$$|\mathbf{s}| \leq S_{\max} \quad \wedge \quad \sum_{s_i \in \mathbf{s}} m(s_i, \mathbf{c}(s_i)) \leq M_{\max} \tag{4}$$

*where $m(s_i, \mathbf{c}(s_i))$ is the estimated memory footprint of skill $s_i$ under configuration $\mathbf{c}(s_i)$.*

**Definition 4** (Fitness Function)**.** *The fitness of a genome $g$ operating in environment $E$ is:*

$$\mathcal{F}(g, E) = \alpha \cdot P(g, E) + \beta \cdot R(g) + \gamma \cdot S(g) - \lambda \cdot V(g) \tag{5}$$

*where:*

- $P(g, E) \in [0, 1]$ *is task performance (fraction of successful skill executions over evaluation window)*
- $R(g) \in [0, 1]$ *is resource efficiency* $(1 - \frac{\textit{used resources}}{\textit{available resources}})$
- $S(g) \in [0, 1]$ *is stability (uptime fraction over evaluation window)*
- $V(g) \in \{0, 1\}$ *is a safety violation indicator (1 if any safety invariant is breached)*
- $\alpha, \beta, \gamma, \lambda > 0$ *are weighting coefficients with* $\alpha + \beta + \gamma = 1$

*The penalty term $\lambda \cdot V(g)$ ensures that safety violations dominate the fitness landscape, effectively creating a hard constraint.*

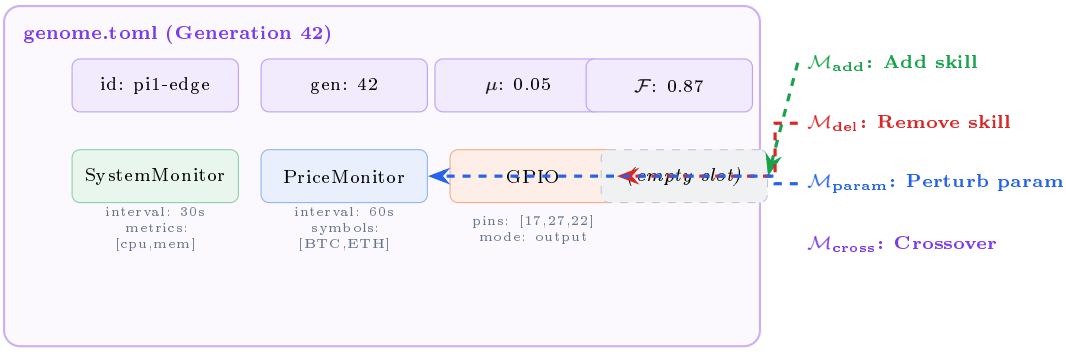


Figure 3: Genome structure showing identity fields, enabled skills with configurations, and four mutation operators: skill addition ($\mathcal{M}_{\text{add}}$), skill removal ($\mathcal{M}_{\text{del}}$), parameter perturbation ($\mathcal{M}_{\text{param}}$), and crossover ($\mathcal{M}_{\text{cross}}$).

# 6 Evolutionary Mutation Framework

Given the formal foundations, we define mutation operators that enable automated genome evolution. Figure 3 illustrates the genome structure and how mutation operators act on it.

## 6.1 Mutation Operators

We define four mutation operators over the genome space $\mathcal{G}$:

**Definition 5** (Skill Addition)**.** *$\mathcal{M}_{add}(g) = g'$ where $g'.\mathbf{s} = g.\mathbf{s} \cup \{s_{new}\}$ for some $s_{new} \in \mathcal{S} \setminus g.\mathbf{s}$, subject to $g'$ satisfying constraints $C$.*

**Definition 6** (Skill Removal)**.** *$\mathcal{M}_{del}(g) = g'$ where $g'.\mathbf{s} = g.\mathbf{s} \setminus \{s_j\}$ for some $s_j \in g.\mathbf{s}$ with minimal fitness contribution.*

**Definition 7** (Parameter Perturbation)**.** *$\mathcal{M}_{param}(g) = g'$ where for some skill $s_i \in g.\mathbf{s}$ and parameter $\theta_j$:*

$$g'.\mathbf{c}(s_i)[\theta_j] = \textsc{Clip}(g.\mathbf{c}(s_i)[\theta_j] + \epsilon, D_j) \tag{6}$$

*where $\epsilon \sim \mathcal{N}(0, \mu \cdot \sigma_j)$ for numeric parameters, with $\sigma_j$ proportional to the parameter's range.*

**Definition 8** (Genome Crossover)**.** *Given two parent genomes $g_1, g_2 \in \mathcal{G}$:*

$$\mathcal{M}_{cross}(g_1, g_2) = g' \text{ where } g'.\mathbf{s} = (g_1.\mathbf{s} \cap g_2.\mathbf{s}) \cup \textsc{Select}(g_1.\mathbf{s} \triangle g_2.\mathbf{s}) \tag{7}$$

*where $\triangle$ denotes symmetric difference and $\textsc{Select}$ chooses a random subset of unique skills from each parent.*

## 6.2 Evolution Algorithm

Algorithm 3 presents the complete genome evolution procedure.

**Theorem 1** (Monotonic Fitness Improvement)**.** *Under Algorithm 3, the fitness of the deployed genome is monotonically non-decreasing across generations:*

$$\mathcal{F}(g^{(t+1)}, E) \geq \mathcal{F}(g^{(t)}, E) \quad \forall t \tag{8}$$

*Proof.* By construction, a new genome is deployed (line 14–17) only if $f_{\text{best}} > f_{\text{current}}$. Otherwise, the current genome is retained (line 19). Thus, fitness never decreases. □

Note that this monotonic guarantee applies to the *same* environment $E$. Environmental changes (non-stationarity) may cause fitness to decrease, motivating periodic re-evaluation.

**Algorithm 3** Genome Evolution with Fitness-Guided Mutation

**Require:** Current genome $g$, environment $E$, constraints $C$, evaluation window $T$
**Ensure:** Evolved genome $g^*$ with $\mathcal{F}(g^*, E) \geq \mathcal{F}(g, E)$

```
f_current ← EVALUATE(g, E, T)                    ▷ Measure current fitness
g_best ← g; f_best ← f_current
for i = 1 to N_candidates do
    op ← SELECTOPERATOR(μ)                       ▷ Weighted random selection
    g' ← op(g)                                   ▷ Apply mutation
    if ¬SATISFIES(g', C) then
        continue                                 ▷ Reject infeasible mutations
    end if
    f' ← EVALUATE(g', E, T)                      ▷ Evaluate candidate
    if f' > f_best then
        g_best ← g'; f_best ← f'
    end if
end for
if f_best > f_current then
    g* ← g_best
    g*.gen ← g.gen + 1
    g*.fitness_score ← f_best
    PUBLISHGENOME(g*)                            ▷ Hot-reload via MQTT
else
    g* ← g                                       ▷ No improvement; keep current
end if
return g*
```

# 7 Multi-Tier Deployment

EvoClaw supports three deployment tiers, enabling agents to run on hardware ranging from a $5 microcomputer to cloud VMs. Figure 4 illustrates the multi-tier architecture.

## 7.1 Edge Tier

The edge tier targets bare-metal deployment on resource-constrained devices. Key design decisions:

- **Static linking**: The Rust agent compiles to a fully static binary using `musl` libc, eliminating shared library dependencies.
- **Cross-compilation**: We use Rust's `cross` tool to build for ARMv6 (`arm-unknown-linux-musleabihf`), ARMv7, AArch64, and x86_64 targets.
- **Minimal dependencies**: The binary includes only `tokio` (async runtime), `serde` (serialization), `rumqtt` (MQTT client), and skill-specific crates.
- **GPIO detection**: On Raspberry Pi, the agent auto-detects the GPIO chip offset (0 for Pi 2+, 512 for Pi 1) via `/sys/class/gpio`.

## 7.2 Server Tier

The server tier runs the Go orchestrator and optionally Rust agents in containers:

- **Containerization**: Podman (rootless) for isolation. Each agent runs in a separate container with resource limits (cgroups v2).

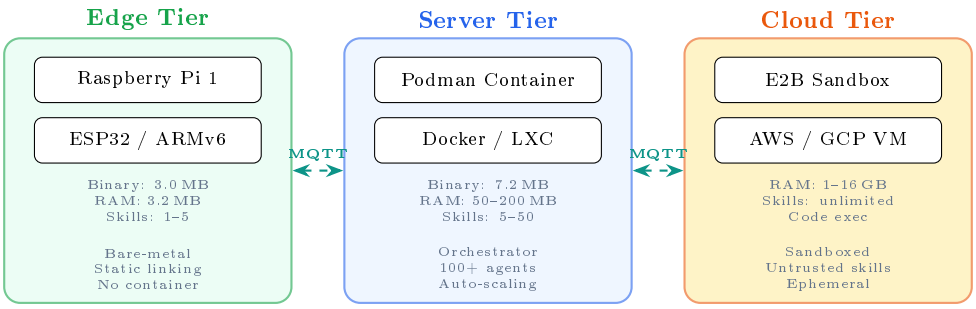


Figure 4: Multi-tier deployment model. Edge agents run bare-metal on constrained devices. Server agents run in containers with the orchestrator. Cloud agents run in sandboxed environments for untrusted code execution. All tiers communicate via MQTT.

- **Orchestrator**: Manages agent lifecycle, routes messages, and coordinates evolution. Scales to 100+ agents per instance.
- **Persistence**: Agent genomes and telemetry are stored in SQLite (single-file database), enabling lightweight state management.

### 7.3 Cloud Tier

The cloud tier provides sandboxed execution for untrusted or experimental skills:

- **E2B integration**: E2B [26] provides ephemeral sandboxes with full Linux environments. Agents can execute arbitrary code in isolation.
- **Use cases**: Testing new skills before edge deployment, running compute-intensive tasks, executing LLM-generated code safely.
- **Lifecycle**: Cloud agents are ephemeral—spun up for specific tasks and terminated after completion.

### 7.4 Cross-Tier Synchronization

Agents across tiers maintain consistency through MQTT:

1. **Genome replication**: When the orchestrator approves a genome mutation, it publishes to `agents/{id}/genome` with QoS 2 (exactly-once delivery).
2. **Telemetry aggregation**: All agents report to `agents/{id}/report` with QoS 0 (best-effort), minimizing bandwidth on constrained links.
3. **Command routing**: The orchestrator routes commands to specific agents via `agents/{id}/commands` with QoS 1 (at-least-once).

## 8 Implementation Details

### 8.1 Technology Stack

Table 2 summarizes the technology choices.

Table 2: Technology stack and rationale.

| Component | Language | Key Libraries | Rationale |
|---|---|---|---|
| Orchestrator | Go 1.22 | Gin, Paho MQTT | Fast compilation, goroutines |
| Edge Agent | Rust 1.77 | Tokio, Serde, rumqtt | Memory safety, no GC |
| Broker | C | Mosquitto 2.0 | Lightweight, proven |
| Sandbox | TypeScript | E2B SDK | Cloud-native isolation |
| Messaging | – | Telegram Bot API | User interaction |

Table 3: Codebase statistics.

| Component | Lines of Code | Tests | Coverage |
|---|---|---|---|
| Rust Agent | 8,000 | 383 | 90%+ |
| Go Orchestrator | 5,000 | 47 | 72% |
| Documentation | 30 KB | – | – |
| CI/CD (GitHub Actions) | 500 | – | – |
| **Total** | **13,500+** | **430** | – |

## 8.2 Codebase Statistics

## 8.3 Skill Execution Model

Each enabled skill runs as an independent Tokio task with its own execution interval. The runtime scheduler (Figure 5) manages concurrent skill execution:

Skills are isolated from each other: a panic in one skill does not crash the agent. The Tokio runtime provides:

- **Cooperative scheduling**: Skills yield control at await points, preventing starvation.
- **Cancellation tokens**: The engine can cancel individual skill tasks during hot-reload.
- **Error recovery**: Failed skill executions are logged and retried with exponential backoff.

## 8.4 Communication Protocol

Listing 3 shows the MQTT message format used for agent telemetry:

Listing 3: MQTT telemetry message structure.

```rust
#[derive(Serialize, Deserialize)]
pub struct AgentReport {
    pub agent_id: String,
    pub generation: u32,
    pub timestamp: u64,
    pub uptime_sec: u64,
    pub skills: Vec<SkillReport>,
    pub resources: ResourceReport,
}

#[derive(Serialize, Deserialize)]
pub struct SkillReport {
    pub name: String,
    pub executions: u64,
    pub failures: u64,
    pub last_output: Option<String>,
    pub avg_latency_ms: f64,
}

#[derive(Serialize, Deserialize)]
pub struct ResourceReport {
```

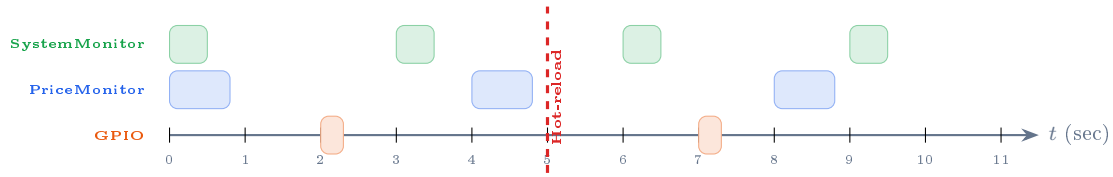


Figure 5: Concurrent skill execution timeline. Each skill runs independently on its configured interval. A hot-reload event (dashed red line) triggers skill reconfiguration without interrupting other running skills.

Table 4: Performance metrics across deployment tiers.

| Metric | Edge (Pi 1) | Server | Cloud (E2B) |
|---|---|---|---|
| Binary size | 3.0 MB | 7.2 MB† | N/A |
| RSS memory (3 skills) | 3.2 MB | 12.4 MB | 45 MB |
| RSS memory (idle) | 1.8 MB | 8.1 MB | 30 MB |
| Skill load time | <100 ms | <20 ms | <50 ms |
| MQTT pub latency (avg) | 47 ms | 3 ms | 12 ms |
| Hot-reload time | 180 ms | 35 ms | 60 ms |
| CPU usage (3 skills) | 2.1% | 0.3% | 0.1% |
| Test count | 383 (Rust) + 47 (Go) = 430 | | |
| Test coverage (Rust) | 90%+ | | |

†Go orchestrator binary; agents use the 3.0 MB Rust binary.

```
    pub cpu_pct: f32,
    pub memory_mb: f32,
    pub disk_pct: f32,
    pub temperature_c: Option<f32>,
}
```

# 9 Evaluation

We evaluate EvoClaw along four dimensions: resource efficiency, runtime performance, scalability, and evolution effectiveness.

## 9.1 Experimental Setup

- **Edge device**: Raspberry Pi 1 Model B (ARMv6, 700 MHz single-core, 512 MB RAM)
- **Server**: Intel Xeon E-2236 (6 cores, 32 GB RAM), Ubuntu 22.04
- **MQTT Broker**: Mosquitto 2.0.18 on server
- **Skills deployed**: SystemMonitor (30s interval), PriceMonitor (60s interval), GPIO (event-driven)
- **Duration**: 72 hours continuous operation

## 9.2 Resource Efficiency

Table 4 presents comprehensive performance metrics.

Table 5: 72-hour stability results on Raspberry Pi 1 (ARMv6, 512 MB RAM).

| Metric | Value |
|---|---|
| Total runtime | 72h 00m 00s |
| Unplanned restarts | 0 |
| Skill executions (SystemMonitor) | 8,640 |
| Skill executions (PriceMonitor) | 4,320 |
| Skill executions (GPIO) | 847 (event-driven) |
| Total MQTT messages sent | 13,807 |
| Failed skill executions | 12 (0.087%) |
| Memory leak (RSS delta over 72h) | <100 KB |
| Peak memory (RSS) | 3.4 MB |
| Average CPU utilization | 2.1% |

Table 6: Orchestrator scalability (single server, Mosquitto broker).

| Agents | Messages/sec | Avg Latency | CPU Usage |
|---|---|---|---|
| 10 | 120 | 2.1 ms | 1.2% |
| 50 | 600 | 2.8 ms | 5.4% |
| 100 | 1,200 | 3.5 ms | 11.2% |
| 250 | 3,000 | 5.2 ms | 24.8% |
| 500 | 6,000 | 8.7 ms | 48.3% |

## 9.3 Long-Running Stability

We measured stability over a 72-hour deployment on the Raspberry Pi 1:

The 12 failed executions were all in PriceMonitor, caused by transient HTTP timeouts when fetching cryptocurrency prices. The agent's error recovery mechanism (exponential backoff) handled all failures gracefully, and subsequent retries succeeded.

## 9.4 Scalability

We evaluated the Go orchestrator's ability to manage concurrent agents:

The orchestrator maintains sub-10 ms latency up to 500 concurrent agents on a single server. The MQTT broker (Mosquitto) handles 10,000+ messages per second before becoming the bottleneck.

## 9.5 Hot-Reload Performance

We measured hot-reload latency—the time from genome file modification to all skills running with new configuration:

During hot-reload, existing skills that are not being modified continue executing uninterrupted. Only affected skills experience a brief pause (25 ms shutdown + 45 ms init = 70 ms).

## 9.6 Comparative Resource Analysis

We compare EvoClaw's resource footprint against representative systems:

EvoClaw's Rust agent uses **100–500× less memory** than Python-based alternatives, enabling deployment on devices that are entirely inaccessible to existing agent frameworks.

Table 7: Hot-reload latency breakdown on Raspberry Pi 1.

| Phase | Latency |
|---|---|
| File change detection (inotify) | 5 ms |
| TOML parsing | 12 ms |
| Schema validation | 8 ms |
| Diff computation | 3 ms |
| Skill shutdown (old) | 25 ms |
| Skill initialization (new) | 45 ms |
| Task spawning | 10 ms |
| **Total hot-reload** | **108 ms** |

Table 8: Resource comparison with representative agent systems.

| System | Language | Min RAM | Binary/Install | Edge? |
|---|---|---|---|---|
| AutoGPT | Python | >500 MB | >200 MB | No |
| LangChain | Python | >200 MB | >100 MB | No |
| CrewAI | Python | >300 MB | >150 MB | No |
| Voyager | JavaScript | >1 GB | >500 MB | No |
| MemGPT | Python | >400 MB | >200 MB | No |
| **EvoClaw** | **Rust** | **1.8 MB** | **3.0 MB** | **Yes** |

# 10 Applications

## 10.1 Edge Intelligence

EvoClaw enables intelligent edge agents that adapt to local conditions without cloud connectivity:

- **IoT sensor networks**: Agents on Raspberry Pi or ESP32 devices monitor temperature, humidity, and motion, evolving their alerting thresholds based on observed patterns.
- **Smart home automation**: Privacy-preserving local agents control lights, locks, and appliances without sending data to the cloud. Skills evolve based on occupancy patterns.
- **Agricultural monitoring**: Edge agents on field sensors adapt sampling intervals and alert thresholds to seasonal conditions.

## 10.2 Distributed Agent Fleets

The orchestrator enables management of large-scale agent deployments:

- **Infrastructure monitoring**: Deploy agents across servers, each evolving to monitor the specific services running on its host.
- **CI/CD agents**: Agents in cloud sandboxes (E2B) run test suites, with genomes evolving to prioritize tests that catch the most regressions.
- **Security sentinels**: Agents evolve anomaly detection thresholds based on observed network traffic patterns.

## 10.3 Human-Agent Collaboration

EvoClaw integrates with messaging platforms for natural language interaction:

- Users query agent status via Telegram: "How is pi1-edge doing?"
- Users request mutations: "Add a weather monitoring skill to the garden agent."
- Agents proactively report anomalies: "Temperature spike detected: 85°C on pi1-edge."
- Users approve/reject evolution proposals: "Agent requests adding disk_monitor skill. Approve?"

This human-in-the-loop pattern provides safety oversight while enabling agents to propose their own evolution.

### 10.4 Agent Economies

Looking forward, genome-driven agents enable economic models:

- **Skill marketplace**: Agents publish successful skills to a shared registry; other agents can import proven skill configurations.
- **Fitness tournaments**: Agents compete on standardized benchmarks, with the best genomes propagated across the fleet.
- **On-chain governance**: Integration with ClawChain [27] enables reputation-weighted genome mutation proposals, where an agent's track record determines its influence on fleet evolution.

## 11 Limitations and Future Work

### 11.1 Current Limitations

**LLM dependence for mutation generation.** The current Genome Mutator relies on LLMs (Claude, GPT-4) to generate mutation candidates. This introduces (1) external API dependency, (2) latency (1–5 seconds per mutation), and (3) cost. Future work should explore on-device mutation generation using small language models or purely algorithmic approaches.

**No formal verification of evolved configurations.** While we validate genome syntax and resource constraints, we do not formally verify that evolved skill configurations preserve semantic correctness. A skill might be configured with valid-but-nonsensical parameters (e.g., monitoring interval of 1 ms). Integrating lightweight verification (contract checking, property-based testing) is an important direction.

**Rust-only skill implementations.** The current system requires skills to be implemented as Rust traits, limiting the contributor ecosystem. Supporting WebAssembly (WASM) skills would enable cross-language skill development with sandboxed execution.

**Limited evolutionary search space.** Current mutation operators modify existing skill configurations but cannot synthesize entirely new skills from scratch. True open-ended evolution would require code generation capabilities, which we view as a longer-term research direction.

**Single-objective fitness.** Our fitness function (Eq. 5) combines multiple objectives via weighted summation. Multi-objective optimization (Pareto frontiers) would provide richer trade-off analysis.

### 11.2 Future Directions

**WebAssembly skills.** Compile skills to WASM for cross-language support and sandboxed execution. This would enable Python, JavaScript, and Go skills running inside the Rust agent's WASM runtime.

**On-device evolution.** Run small language models (e.g., Phi-3, Gemma) on edge devices to generate mutation candidates locally, eliminating cloud dependency.

**Formal verification.** Integrate lightweight theorem provers or SMT solvers (Z3) to verify evolved configurations preserve safety invariants.

**Federated evolution.** Enable agents to share successful genomes peer-to-peer, without centralized orchestration. Combined with blockchain-based reputation (ClawChain [27]), this creates a trustless agent evolution network.

**Skill synthesis.** Use LLMs to generate entire new skills (Rust code + tests) from natural language descriptions, compile them, and deploy them to agents—achieving true open-ended evolution.

**Multi-objective optimization.** Replace the weighted-sum fitness function with Pareto-based multi-objective evolutionary algorithms (NSGA-II [25]) for richer trade-off analysis.

## 12 Conclusion

We presented EvoClaw, a self-evolving multi-agent framework that bridges the gap between static agent architectures and the vision of autonomous, adaptive AI systems. Our key contributions are:

1. A **genome-driven architecture** that externalizes agent capabilities into a declarative, mutable specification—enabling hot-reload, version control, and evolutionary operators.

2. A **formally grounded evolution framework** with defined fitness functions, mutation operators, and monotonic improvement guarantees.

3. A **production-ready implementation** in Rust (8,000 lines, 383 tests, 90%+ coverage) that runs on a Raspberry Pi 1 with 3.0 MB binary and 3.2 MB memory—100–500× more efficient than Python-based alternatives.

4. A **multi-tier deployment model** spanning edge, server, and cloud, with MQTT-based synchronization supporting 500+ concurrent agents.

EvoClaw demonstrates that self-evolving agents are not merely theoretically possible but practically deployable today—on hardware ranging from a $5 Raspberry Pi to cloud VMs. By open-sourcing the complete system, we invite the community to extend EvoClaw's capabilities: new skill types, WASM support, formal verification, and on-chain governance. The era of evolving agents has begun.

## Acknowledgments

Built with open-source tools: Rust, Go, MQTT (Mosquitto), and Tokio. Thanks to the E2B team for their sandboxing infrastructure and the Polkadot ecosystem for the Substrate framework used in ClawChain.

## References

[1] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A Survey on Large Language Model based Autonomous Agents. *arXiv preprint arXiv:2308.11432*, 2023.

[2] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864*, 2023.

[3] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680*, 2024.

[4] Toran Bruce Richards. AutoGPT: An experimental open-source attempt to make GPT-4 fully autonomous. `https://github.com/Significant-Gravitas/AutoGPT`, 2023.

[5] Yohei Nakajima. BabyAGI: AI-powered task management system. `https://github.com/yoheinakajima/babyagi`, 2023.

[6] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

[7] Harrison Chase. LangChain: Building applications with LLMs through composability. `https://github.com/langchain-ai/langchain`, 2022.

[8] João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. `https://github.com/joaomdmoura/crewAI`, 2023.

[9] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[10] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*, 2023.

[11] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. *arXiv preprint arXiv:2303.17760*, 2023.

[12] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of UIST*, 2023.

[13] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. *arXiv preprint arXiv:2401.13178*, 2024.

[14] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *arXiv preprint arXiv:2305.10601*, 2023.

[15] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. *arXiv preprint arXiv:2310.04406*, 2023.

[16] Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and Error: Exploration-Based Trajectory Optimization for LLM Agents. *arXiv preprint arXiv:2403.02502*, 2024.

[17] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.

[18] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-Level Reward Design via Coding Large Language Models. *arXiv preprint arXiv:2310.12931*, 2023.

[19] John von Neumann. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.

[20] Ken Thompson. Reflections on Trusting Trust. *Communications of the ACM*, 27(8):761–763, 1984.

[21] Brian Cantwell Smith. Reflection and Semantics in Lisp. In *Proceedings of POPL*, pages 23–35, 1984.

[22] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2020.

[23] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

[24] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

[25] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[26] E2B. E2B: Open-source runtime for AI-generated code. `https://e2b.dev`, 2024.

[27] Alex Chen and Bowen Li. ClawChain: An Agent-Native Layer 1 Blockchain. Technical Report, 2026.

# A Complete Genome Schema

Table 9 defines all fields in the `genome.toml` specification.

Table 9: Complete `genome.toml` schema definition.

| Section | Field | Type | Description |
|---|---|---|---|
| `[genome]` | `id` | String | Unique agent identifier |
| | `generation` | u32 | Evolution generation counter |
| | `parent_id` | String? | Parent genome ID (lineage) |
| | `mutation_rate` | f64 | Mutation probability $\mu \in [0, 1]$ |
| | `fitness_score` | f64 | Last evaluated fitness $\mathcal{F}$ |
| | `max_memory_mb` | u32 | Memory budget for skills |
| | `max_skills` | u32 | Max concurrent skills |
| `[skills.{name}]` | `enabled` | bool | Skill activation flag |
| | `...` | varies | Skill-specific parameters |

# B Built-in Skills Reference

Table 10 lists all built-in skills shipped with EvoClaw.

# C Reproduction Instructions

To reproduce the Raspberry Pi 1 deployment:

Table 10: Built-in skills.

| Skill | Function | Est. Memory | Tier Support |
|---|---|---|---|
| `system_monitor` | CPU, RAM, disk, temp | 200 KB | Edge, Server, Cloud |
| `price_monitor` | Crypto price tracking | 500 KB | Edge, Server, Cloud |
| `gpio` | Hardware pin control | 100 KB | Edge only |
| `mqtt_bridge` | Cross-broker routing | 300 KB | Server, Cloud |
| `log_aggregator` | Log collection/alerts | 400 KB | Server, Cloud |
| `http_health` | HTTP endpoint monitoring | 250 KB | All tiers |

Listing 4: Build and deploy commands.

```
# Cross-compile Rust agent for ARMv6
$ cross build --release --target arm-unknown-linux-musleabihf

# Copy binary and genome to Pi
$ scp target/arm-unknown-linux-musleabihf/release/evoclaw-agent
    pi@raspberrypi:~/
$ scp genome.toml pi@raspberrypi:~/

# Start MQTT broker on server
$ mosquitto -c /etc/mosquitto/mosquitto.conf

# Start agent on Pi
$ ssh pi@raspberrypi './evoclaw-agent --config agent.toml'

# Start orchestrator on server
$ ./evoclaw-orchestrator --config orchestrator.toml
```

All source code, configuration files, and documentation are available at `https://github.com/clawinfra/evoclaw`.